

AWS Production Architecture & DevOps Proposal

Intelligent Technologies Ltd. — Technical Review Document
Prepared by: Eng. Fareed Mohamed
To: Eng. Riyadh

1. Architecture Overview

Figure 1. AWS Production Architecture

The proposed architecture is designed to provide a simple, secure, and cost-effective production environment for a small-scale website. User requests reach the environment through an optional Route 53 DNS record, pass through a Security Group that restricts inbound access, and are handled by a single EC2 (Ubuntu) instance. The instance runs Docker, hosting an Nginx reverse proxy alongside the application container. CloudWatch provides monitoring for the instance.

2. CI/CD Pipeline

Figure 2. CI/CD Pipeline

Developers push source code to the GitHub Repository. GitHub Actions automatically builds the application, builds a Docker image, performs a Trivy security scan, and deploys to the EC2 server over SSH. On the server, the pipeline pulls the latest image, stops the old container, and runs the new one. A health check determines whether the deployment succeeded: on success traffic is served from Production, on failure the pipeline triggers a Rollback. GitHub Actions was selected for this pipeline because the team already uses GitHub, it integrates directly with the repository, requires no dedicated CI server, and is more cost-effective than maintaining Jenkins for this project.

3. Deployment and Rollback Workflow

Figure 3. Deployment and Rollback Workflow

During deployment, the new container (V2) is started while the current container (V1) remains available and serving traffic. A health check verifies the new container. If the check passes, V1 is removed and V2 becomes the production container. If the check fails, V2 is stopped, V1 is restarted, and the deployment is automatically rolled back.

4. AWS Services

Service	Purpose
Route 53 (optional)	DNS management
Security Group	Restrict inbound network access
EC2 (Ubuntu)	Host the application via Docker
CloudWatch	Monitoring and logs

5. DevOps Tools

Tool	Purpose
GitHub	Source Code Management
GitHub Actions	CI/CD Automation
Docker	Containerization

Trivy	Container Security Scanning
Terraform	Infrastructure as Code (optional)

Note: Terraform is optional for this project and is recommended only if infrastructure provisioning needs to be repeatable or managed as code.